PYTHON · BANCO DE DADOS

Peewee: do objeto ao dado

Como classes Python se transformam em persistência relacional

Modelos em memória gravados com segurança no disco.

Mapeamento Objeto-Relacional na prática

João Rafael, Matheus Medeiros, Artur Bezerra e Heitor Gusmão



MODELO CONCEITUAL

ORM traduz classes em tabelas

O Peewee vincula atributos de instância a colunas de banco relacional, automatizando tipos, chaves e restrições estruturais.

Classe Amigo

Herda de `Model` e define o esquema na memória do programa.

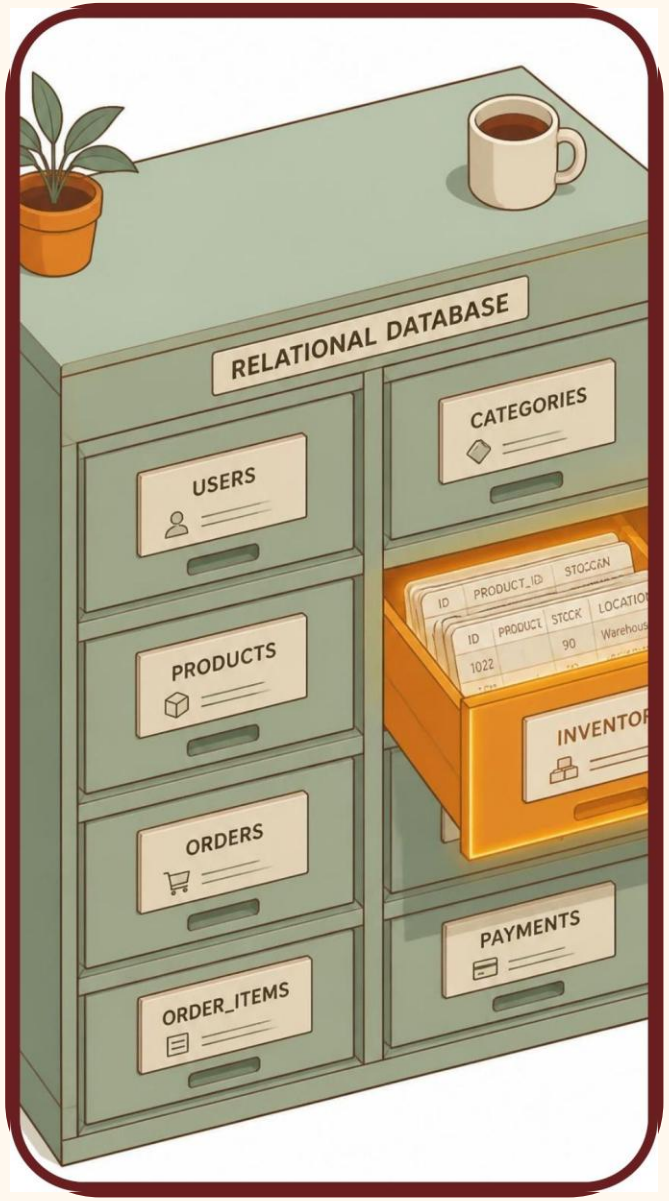
Atributos e tipos

Campos como `CharField` e `AutoField` mapeiam tipos nativos.

Tabela relacional

Cada linha representa uma instância persistida com identificador único.

Exemplo-guia: entidade com id numérico, nome e usuário único

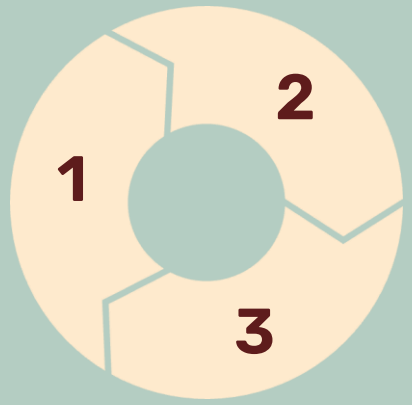


FLUXO OPERACIONAL

CRUD organiza o ciclo principal dos dados

A interação entre a aplicação Python, a camada do Peewee e o motor de banco de dados acontece em um circuito contínuo.

Aplicação Python



Camada do Peewee

Banco de dados

Criar

`create()`, `save()`, `insert_many()`

Consultar

`select().where()`, `get()`, `get_or_none()`, `get_or_create()`

Atualizar e remover

`save()`, `update().execute()`, `delete_instance()`

PERSISTÊNCIA INICIAL

Inserir depende do cenário

A escolha entre operações pontuais ou em lote equilibra simplicidade, controle prévio do objeto e desempenho de I/O.

OPERATÓRIO DIRETO

create()

Instancia e grava em uma única instrução SQL.

Ideal quando todos os valores já estão validados e prontos para salvar de imediato.

ETAPAS SEPARADAS

instância + save()

Cria o objeto em memória antes de persistir.

Permite inspecionar, alterar ou enriquecer atributos antes de enviar o INSERT ao banco.

ALTO VOLUME

insert_many()

Carrega listas de dicionários com alto ganho de velocidade.

Atenção: essa rota em lote ignora os sinais automáticos de ciclo de vida de instância.

O ganho de throughput com insert_many() exige validação externa prévia

BUSCA RIGOROSA

Esperando registro único

Quando o código assume que o dado precisa existir.

MÉTODO	`Amigo.get()`
COMPORTAMENTO	Retorna 1 objeto exato.
CASO AUSENTE	Dispara exceção `DoesNotExist`.
USO IDEAL	Busca por chave primária garantida.

BUSCA DEFENSIVA

Tratando a ausência

Quando a ausência é um estado esperado da regra.

MÉTODO	`get_or_none()` e `where()`
COMPORTAMENTO	Retorna None ou queryset iterável.
PREVENÇÃO	`get_or_create()` evita duplicatas.
USO IDEAL	Consultas de busca e telas de login.

CONTROLE DE ESCOPO

Atualizar e remover exigem precisão

Diferenciar operações em instâncias carregadas de comandos executados direto no banco previne alterações acidentais em cascata.

save()

Instância pontual

Carrega o registro, altera o atributo em Python e grava. Dispara eventos de ciclo de vida com rastreio total.

where()

update().execute()

Altera em massa direto nas linhas. **Atenção:** sem a cláusula `where()`, a modificação atinge toda a tabela.

delete()

delete_instance()

Remove a linha da instância já localizada e dispara o hook `pre_delete` antes da destruição.

Nunca execute comandos de atualização estrutural sem delimitar filtros explícitos.

Validação no código não substitui integridade no banco

Validação

- Nome e usuário não podem ficar vazios
- O usuário deve ser único

Integridade

A validação em Python ajuda, mas `unique=True` protege contra duplicidade em operações concorrentes

ARQUITETURA DE AUDITORIA

Sinais transformam operações em eventos auditáveis

Hooks de ciclo de vida desacoplam regras de negócio, permitindo validar e registrar alterações sem poluir a rotina principal.

01

pre_save

Interrompe e valida o objeto antes do envio ao banco. Permite sanitizar campos ou normalizar strings.

02

post_save

Inspeciona a flag `created` para distinguir novos cadastros de atualizações e gerar logs de auditoria.

03

pre_delete

Captura os dados da instância antes da exclusão física, permitindo arquivamento ou histórico de remoção.

Lembrete essencial: operações em lote com `insert_many()` ou `update()` não disparam sinais de instância.

TRATAMENTO DE FALHAS

Exceções tornam o fluxo previsível

O bloco `try/except` traduz anomalias técnicas em respostas claras para o cliente sem interromper a execução do serviço.

EXCEÇÃO	CAUSA RAIZ NO FLUXO	TRATAMENTO RECOMENDADO
DoesNotExist	Consulta com `get()` não localizou nenhuma linha.	Retornar status 404 ou mensagem amigável de registro não encontrado.
IntegrityError	Violação de `unique=True` ou chave estrangeira.	Alertar duplicidade de usuário ou campo nulo proibido.
ValueError	Dados inconsistentes detectados pela validação lógica.	Interromper antes do banco e solicitar correção nos parâmetros enviados.

Mapear exceções explícitas impede falhas silenciosas e vazamento de stack traces internos

Persistência fecha o ciclo

Objeto e banco

O objeto começa na memória e passa a existir no banco após save().

Reinício

Ao reiniciar o programa, o registro continua recuperável se o mesmo banco for usado.

Testes

Teste o caminho feliz e os casos inválidos: duplicidade, ausência, atualização, exclusão e reinício.

